

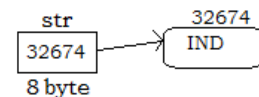
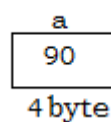
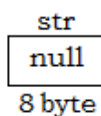
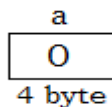
5. Variables

- ❖ Variable is the container which holds user data.
- ❖ Memory will be allocated for the variable while executing the program.
- ❖ Value of the variable can be changed any number of times during the program execution.

Syntax

```
<Data type> <varName>;
<Data type> <varName>=<values>;
```

Example: - 1) int a; 2) String str; 3) int a=99; 4) String str="IND";



Type of Variable:-

Based on Data type, there are two type of variable.

1. Primitive Variable
2. Reference Variable

1. Primitive Variable

- ❖ Variable declared with the primitive data type (like Boolean, byte, char, short, int, long, double, float) are called as primitive variable.
- ❖ Memory allocation for primitive variables depends on the primitive data type.
- ❖ Default value of primitive variables depends on primitive data type.
- ❖ Primitive variables hold valid literals or value in the allocated memory.

Example:

```
int a;
double d=4.2;
```

2. Reference Variable

- ❖ Variable declared with user define data type (like String) are called as Reference variable.
- ❖ Always 8 bytes of memory will be allocated for Reference variables
- ❖ Default value of Reference variables is always null.
- ❖ Primitive variables hold either null or address of Object in the allocated memory.

Example:

```
String str1;
String str2= "HELLO";
```

First Simple Java Program:

```
class FirstProgram {
public static void main(String args[]){
    System.out.println("Hello, Welcome to Java");
}
}
```

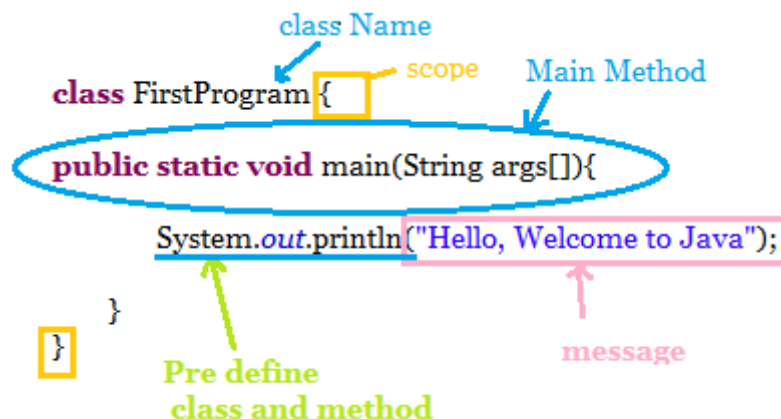
Output

Hello, Welcome to Java

Java program to check default value:

JAVA1.java <pre> class Hello{ boolean b1; byte b2; short s; int a; long b; float f; double d; String str; Hello h; char ch; void show(){ System.out.println(b1); System.out.println(b2); System.out.println(s); System.out.println(a); } } </pre>	<pre> System.out.println(b); System.out.println(f); System.out.println(d); System.out.println(ch); System.out.println(str); } } class JAVA1{ public static void main (String arg[]) { Hello h =new Hello(); h.show(); } } </pre>
JAVA2.java <pre> class Hello{ char ch; void show(){ System.out.println(ch==0); System.out.println(ch==' '); System.out.println(ch=="\u0000"); } } </pre>	<pre> class lab2{ public static void main (String arg[]){ Hello h = new Hello(); h.show(); } } </pre>

Program details : -



Type of Variable:-

Based on Scope, There are three type of variable.

1. Instance Variable
2. Static Variable
3. Local Variable

1. Instance Variable

- ❖ Variable declare in the class without static modifier is called as Instance Variable.
- ❖ JVM is providing default value for Instance variable.
- ❖ Instance variable cannot be accessed directly from Static method (like Main method).

1. Static Variable

- ❖ Variable declare in the class with static modifier is called as Static Variable.
- ❖ JVM is providing default value for Static variable.
- ❖ Static variable can be accessed from Static method (like Main method).

1. Local Variable

- ❖ Variable declare within the member of class (like Method etc) called as Local Variable.
- ❖ JVM is **not** providing default value for Local variable.
- ❖ Local variable must be initialized before using.
- ❖ If Local variable declared but not initialized and not used then that variable declaration statement will be removed by Compiler while translating to byte code. (To verify this, open class file with “DCompiler”) (“DCompiler” is easily available on internet)

```

class Hello {
    int a;      → Instance Variable
    static int b; → Static Variable
    void show(){
        int c; → Local Variable
    }
}

```

Declaration and Initialization of Variable:**1. Declaration of variable in one variable declaration statement.**

Syntax:

[Modifiers] <Data Type> <varName>;

Example:

```

int i;
byte b;
String st;    etc.

```

Example:**JAVA3.java**

```

class JAVA3{
public static void main (String args []){
int a;
System.out.println("Hello");
}
}

```

JAVA4.java

```

class JAVA4{
public static void main (String args []){
int a;
System.out.println(a);
}
}

```

JAVA5.java

```

class JAVA5{
static int a;
public static void main (String args []){
System.out.println(a);
}
}

```

JAVA6.java

```

class JAVA6{
int x;
public static void main (String args []){
System.out.println(x);
}
}

```

Initialization of variable in another statement.

Syntax:

<varName>=<value/expression>;

Example:

```

int i;
i=10;
byte b;
b=20;
String st;
st= "OM"      etc.

```

JAVA7.java

```

class JAVA7{
public static void main (String args []){
String str;
str="IND";
System.out.println(str);
}
}

```

JAVA8.java

```

class JAVA8{
public static void main (String args []){
int a;
a=99;
System.out.println(a);
}
}

```

2. Declaration and Initialization A single variable in one variable declaration statement.

Syntax:

[Modifiers] <Data Type> <varName>=<value/expression>;

Example:

```

int i=10;
byte b=20;
String st= "IND";  etc.

```

JAVA9.java

```

class JAVA7{
public static void main (String args []){
int a=10;
boolean b=true;
double d =99.99;
String str = "IND";
System.out.println(a);
System.out.println(b);
System.out.println(d);
System.out.println(str);
}
}

```

3. Declaring multiple variables of single type in one variable declaration statement.

Syntax:

[Modifiers] <Data Type> <var1>, <var2>, <var3>..;

Example:

```
int a, b, c;
byte b1, b2, b3;
```

Initializing multiple variables with same value;

Syntax:

[Modifiers] <Data Type> <var1>, <var2>, <var3>..=<value/Expression>;

Example:

```
int a, b, c;
a=b=c=99;
String st1, st2, st3;
st1= st2= st3= "Java";
```

JAVA10.java

```
class JAVA10{
public static void main (String args []){
int a,byte b;
System.out.println(a);
System.out.println(b);
}
}
```

JAVA11.java

```
class JAVA11{
public static void main (String args []){
int a,b,c;
a=b=c=99;
System.out.println(a);
System.out.println(b);
System.out.println(b);
}
}
```

JAVA12.java

```
class JAVA12{
public static void main (String args []){
String str1, str2, str3;
str1=str2=str3="IND";
System.out.println(str1);
System.out.println(str2);
System.out.println(str3);
}
}
```

4. Declaring and initializing multiple variables of single type in one variable declaration statement.

Syntax:

[Modifiers] <Data Type> <var1>=<val1>, <var2>=<val2>, ...

Example:

```
int a=10, b=20, c=30;
a=b=c=99;
String st1= "Hi", st2="Java", st3="is Simple";
boolean b1=true; boolean b2=false;
```

JAVA13.java

```
class JAVA13{
public static void main (String args []){
String str1="IND",srt2="DELHI";
System.out.println(str1);
System.out.println(srt2);
}
}
```

JAVA14.java

```
class JAVA14{
public static void main (String args []){
int a=120, b=76, c=56;
System.out.println(a);
System.out.println(b);
System.out.println(c);
}
}
```

JAVA15.java

```
class JAVA15{
public static void main (String args []){
String str1, srt2, str3;
str1=srt2=str3="IND";
System.out.println(str1);
System.out.println(srt2);
System.out.println(str3);
}
}
```

JAVA16.java

```
class JAVA16{
public static void main (String args []){
int a=b=c=50;
System.out.println(a);
System.out.println(b);
System.out.println(c);
}
}
```

JAVA17.java

```
class JAVA17{
public static void main (String args []){
int b, c;
int a=b=c=50;
System.out.println(a);
System.out.println(b);
System.out.println(c);
}
}
```

JAVA18.java

```
class JAVA18{
public static void main (String args []){
int a, b, c;
a=b=c=99;
System.out.println(a);
System.out.println(b);
System.out.println(b);
}
}
```

JAVA19.java

```
class JAVA19{
public static void main (String args []){
int a=a=99;
System.out.println(a);
}
}
```

JAVA20.java

```
class JAVA20{
public static void main (String args []){
int a=b, b=99;
System.out.println(a);
System.out.println(b);
}
}
```

Program output /error and details:-

Prog No.	output	error and details
JAVA1	false 0 0 0 0 0.0 0.0 0	Here, Variable declared with the Primitive and Reference data type and also Variables are Instance so that it will print default value.

	null	
JAVA2	true false true	Default value of char is either ASCII – o or UNICODE \u0000. Space is not a default value of char.
JAVA3	Hello	If Local variable declared but not initialized (like 'a') and not used then that variable declaration statement will be removed by Compiler while translating to byte code.
JAVA4	Compilation error	[Error: Non-Static variable might not be initialise] JVM is not providing default value for Local variable.
JAVA5	0	JVM is providing default value for Static variable.
JAVA6	Compilation error	[Error: Cannot make a static reference to the non-static field x] JVM is providing default value for Instance variable but without creating object we can't call Instance variable.
JAVA7	IND	Here, first variable is declared in one statement and that variable is initializing in 2 nd statement.
JAVA8	99	same ""
JAVA9	10 true 99.99 IND	Here, Variable is declared and initialised in same statement.
JAVA10	Compilation error	[Error: 'b' not a statement] You can't declare more than one variable in same statement.
JAVA11	99 99 99	You can initialise multiple variables with same value like that only.
JAVA12	IND IND IND	same ""
JAVA13	IND DELHI	You can declared and initialise multiple variable of single type in one statement.
JAVA14	120 76 56	same ""
JAVA15	IND IND IND	Here, str1 and str2 are declared in one statement and str3 is declared and all variable are initialized in another statement.
JAVA16	Compilation error	[Error: variable b, c are not declared] only variable 'a' is declared.
JAVA17	50 50 50	You can declared and initialise multiple variable of single type in one statement.
JAVA18	Compilation error	[Error: Duplicate local variable b, c] b and c are declare two time.
JAVA19	99	Here, only one variable 'a' is declared.
JAVA20	Compilation error	[Error: variable b is not declared]

6. Constant

- ❖ Constant is a variable whose value can't be modified during the program execution.
- ❖ Constant is also called as final variable.

Syntax:

```
[Modifier] final <data Type> <varName>=<Value>;
```

Example:

```
final int A=99;
final String str= "IND";
```

- ❖ A variable that is declared as final not initialized is called as **blank final variable**.

Syntax:

```
[Modifier] final <data Type> <varName>;
```

Example:

```
final int A;
A=88;
```

- ❖ We can't use const keyword to declare constant.

Program:

JAVA21.java

```
class JAVA21{
public static void main (String args []){
int A=99;
System.out.println(A);
A=88;
System.out.println(A);
}
}
```

JAVA22.java

```
class JAVA22{
public static void main (String args []){
final int A=99;
System.out.println(A);
A=88;
System.out.println(A);
}
}
```

JAVA23.java

```
class JAVA23{
public static void main (String args []){
final int a;
    System.out.println("Hello");
}
}
```

JAVA24.java

```
class JAVA24{
public static void main (String args []){
final int a;
    System.out.println(a);
}
}
```

JAVA25.java

```
class JAVA25{
public static void main (String args []){
final int a;
a=99;
System.out.println(a);
}
}
```

JAVA26.java

```
class JAVA26{
public static void main (String args []){
final int x;
x=99;
System.out.println(x);
x=99;
System.out.println(x);
}
}
```


JAVA27.java

```
class JAVA27{
public static void main (String args []){
const int x=88;
    System.out.println(x);
}
}
```

JAVA28.java

```
class JAVA28{
public static void main (String args []){
final int a=60;
    System.out.println(a);
    System.out.println(a+90);
}
}
```

JAVA29.java

```
class JAVA29{
public static void main (String args []){
final String str ="IND";
    System.out.println(str);
}
}
```

JAVA30.java

```
class JAVA30{
public static void main (String args []){
final String str ="IND";
str="Welcome";
    System.out.println(str);
}
}
```

Program output /error and details:-

Prog No.	output	error and details
JAVA21	99 88	'a' is local variable. you can assigned local variable many time with deferent value
JAVA22	Compilation error	[Error: The final local variable 'A' cannot be assigned]
JAVA23	Hello	If Local final variable declared but not initialized (like 'a') and not used then that variable declaration statement will be removed by Compiler while translating to byte code.
JAVA24	Compilation error	[Error: The final local variable 'a' may not have been initialized]
JAVA25	99	it is blank final variable.
JAVA26	Compilation error	[Error: The final local variable 'x' may already have been assigned]
JAVA27	Compilation error	[Error: Syntax error on token "const", delete this token]
JAVA28	60 150	We can add value with final variable and hard coded value only.
JAVA29	IND	it is final reference variable.
JAVA30	Compilation error	[Error: The final local variable str cannot be assigned. It must be blank and not using a compound assignment]